Data Visualization — Task 1 (A & B)
Japan Inbound Immigration Dashboard

Mustafa Bozyel 32417

March 23, 2026

GitHub repository: https://github.com/mufasa-349/Data-Visualization-HW2.git

Abstract

This report describes Task 1 of the homework: building an interactive Python dashboard for Japan inbound immigration statistics, analyzing the interface with Gestalt and Bertin principles, and documenting revisions made to improve visual encoding and readability. Screenshots of the dashboard *before* and *after* revision are included, together with a structured comparison of analytical findings, implemented changes, and *user-visible benefits* of the revised interface. Program code for the dashboard and configuration was produced with assistance from **Cursor**, used as the AI coding agent in the development workflow.

1 What we did (overview)

1.1 Task A — Interactive dashboard

- **What:** We visualized the dataset `japan_immigration_statistics_inbound.csv` (Japan inbound immigration, annual records from 1950–2005) using an interactive web dashboard.
- **How:** We used **Python** with **Streamlit** for layout, filters, and metrics, and **Plotly** for charts (time series, stacked areas by continent, country rankings, and a heatmap). Dependencies are listed in `requirements.txt` (`streamlit`, `pandas`, `plotly`). A virtual environment is used for installation.
- **AI-assisted coding (Cursor):** Implementation files—including `dashboard.py`, `requirements.txt`, and `.streamlit/config.toml`—were written and revised with **Cursor** acting as the **AI agent** for code generation: the assistant produced initial layouts, Plotly figures, theme helpers, bug fixes (e.g., compatibility with Plotly's API), and Streamlit API updates. The author reviewed, ran, and validated all code locally.
- **Deliverables:** Runnable application (`dashboard.py`), configuration (`.streamlit/config.toml` for theme), data file, and screenshots (`task1-a.png`, `task1-b.png`).

1.2 Task B — Gestalt & Bertin analysis

- **What:** We analyzed the dashboard using **Gestalt** grouping principles (proximity, similarity, continuity, figure–ground, common region) and **Bertin's** visual variables (especially position, size, color/value, and redundant encoding).
- **How:** Findings were written in `task1-b-gestalt-bertin-analysis.txt`. The same criteria guided UI revisions in `dashboard.py`.

- **Outcome:** The report below merges that analysis with a **before/after** narrative: Figure 1 reflects the pre-revision UI; Section 4 summarizes changes aligned with the checklist; Figure 2 shows the revised dashboard; Section 6 states explicitly *what is better* after revision and why.

2 Dashboard screenshot (before principles-based revision)

Figure 1 shows the initial dashboard (Time series tab), which was used as the basis for the Gestalt/Bertin critique. At this stage, the layout already used a dark theme, sidebar filters, KPI row, and two charts (annual total line chart and year-over-year percentage bars).

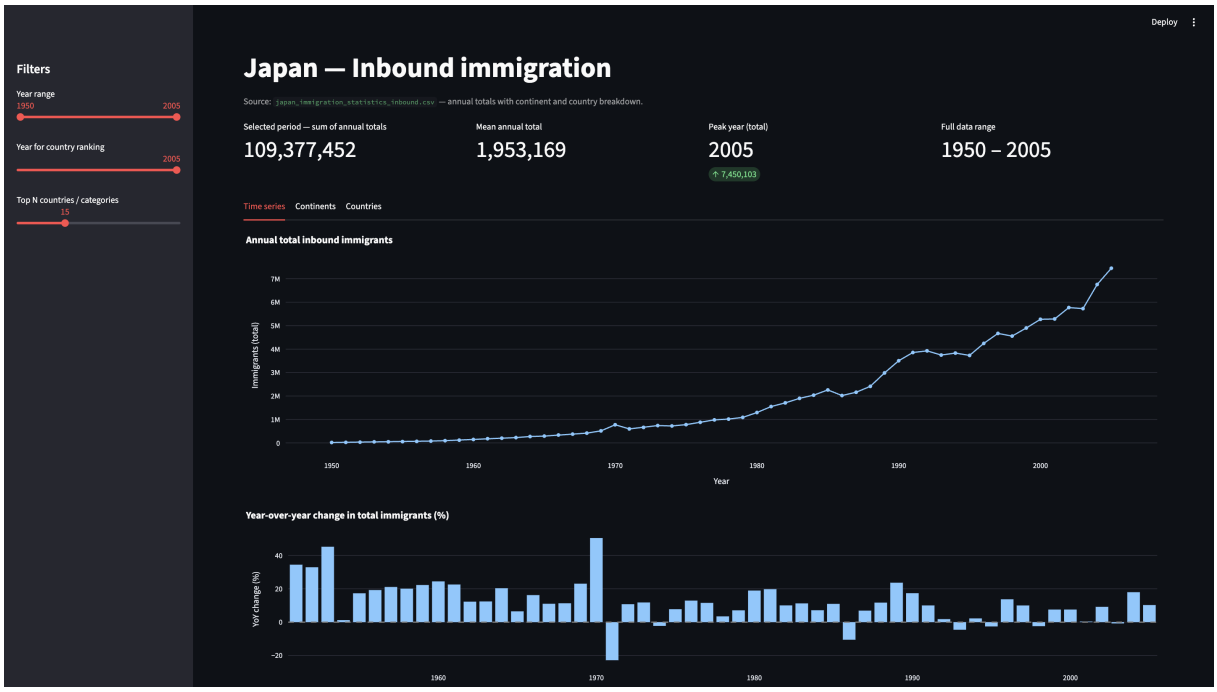

Figure 1: Initial interactive dashboard (before revision): Japan inbound immigration — Time series view with KPIs, annual total line chart, and YoY change bar chart.

3 Gestalt and Bertin analysis (baseline — before revision)

This section summarizes the written analysis in `task1-b-gestalt-bertin-analysis.txt`, describing the **pre-revision** interface.

3.1 Gestalt — strengths

- **Proximity:** Filters are grouped in the sidebar; KPI metrics sit in one row, so controls and summary numbers form coherent units.
- **Similarity:** Both charts use the same hue family (blue) for data, signaling one phenomenon (immigration counts and their change).
- **Continuity:** The line chart supports a clear temporal reading from 1950 to 2005.
- **Figure–ground:** On a dark background, light chart elements and text separate well from the panel.
- **Common region:** The main content is visually separated from the sidebar (controls vs. readout).

3.2 Gestalt — weaknesses

- **Similarity / semantics:** Red accents (sliders, active tab) vs. green KPI "delta" mix navigation and magnitude cues; users may briefly confuse "red" with negative data.
- **Continuity / YoY:** The YoY chart omits the first year in the filtered range (no prior row); a short caption was missing.
- **Proximity:** Four KPIs in one row can feel dense on narrow layouts; definitions were not always explicit.
- **Closure:** The total series could use a subtle area under the line to reinforce cumulative "volume" over time.

3.3 Bertin — strengths

- **Position:** Time on the horizontal axis and magnitude on the vertical axis follows effective encoding priority.
- **Size (typography):** Title, KPI numerals, and axis ticks establish a reasonable hierarchy.
- **Color value:** Light blue on dark background works for the main series and bars.

3.4 Bertin — weaknesses

- **Redundant encoding:** YoY sign was carried mainly by position above/below zero; hue could reinforce positive vs. negative years.
- **Mark size / noise:** Markers at every year on the line chart add visual noise on long series.
- **Units and definitions:** Axis titles and subtitles should state units and the meaning of `total` explicitly.
- **Contrast:** Axis tick labels on dark theme should meet comfortable contrast (WCAG-oriented).

3.5 Summary (pre-revision)

Strengths included clear temporal layout (Bertin: position), coherent grouping (Gestalt: proximity), and strong figure–ground separation. Gaps included redundant encoding for signed YoY change, clearer semantics for accent colors, and richer annotations (units, first-year YoY behavior).

4 Principles-based revisions (what we implemented)

Table 1 maps the actionable checklist from the analysis file to concrete changes in code and configuration.

5 Dashboard screenshot (after revision)

Figure 2 shows the revised dashboard. Visually, the total series now includes a filled area under the line; YoY bars use color to encode sign; typography and grids are harmonized; and textual annotations (subtitles, captions, metric help) clarify definitions and the first-year YoY behavior.

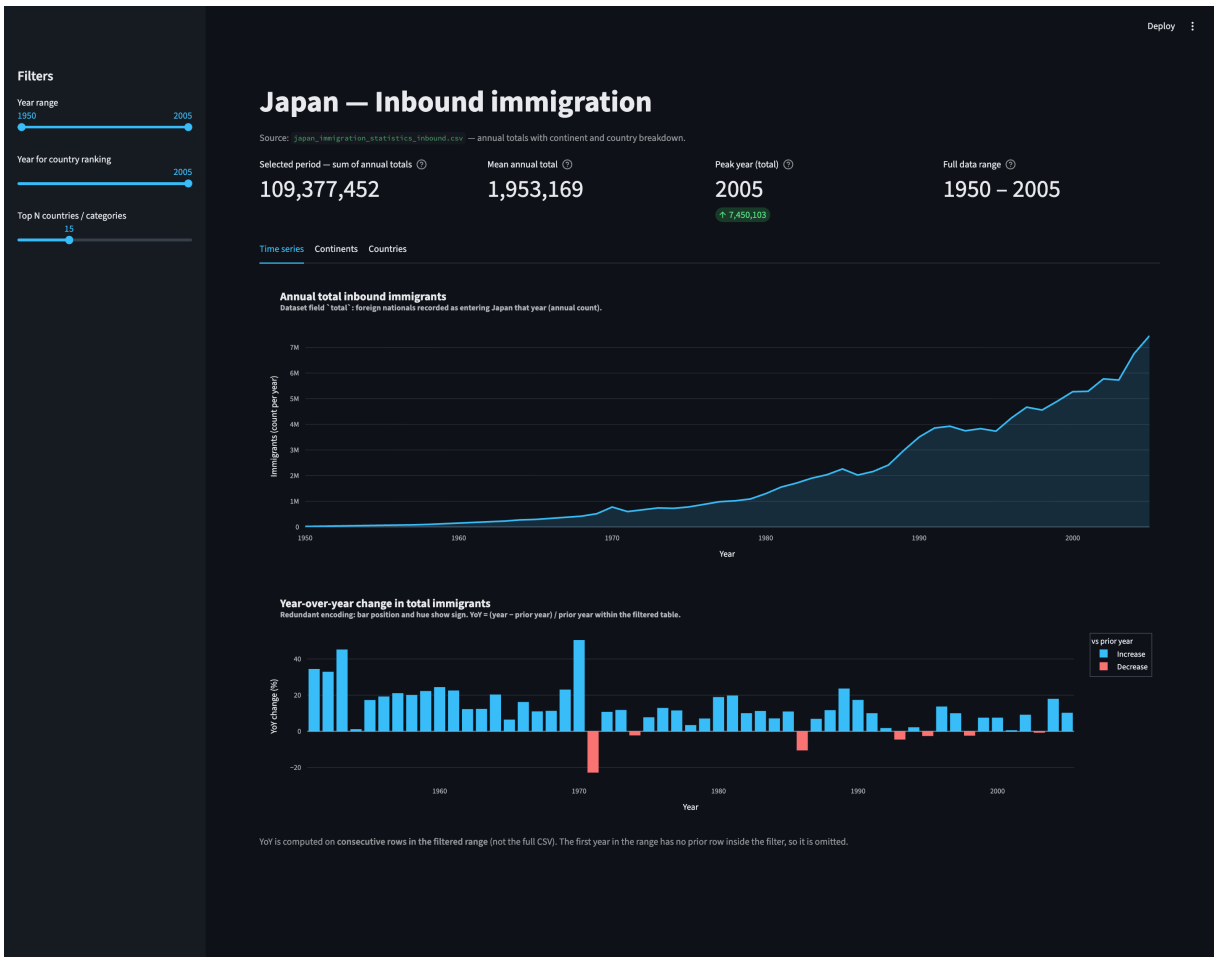

Figure 2: Revised interactive dashboard (after Gestalt/Bertin–aligned changes): unified theme, filled area under the annual total, redundant hue encoding for YoY bars, improved axis/tick contrast, and explanatory subtitles/captions.

Principle / goal	Implementation
Chart subtitles & definition of `total`	Subtitles under chart titles (Plotly `title` HTML); axis labels such as "Immigrants (count per year)".
YoY redundant encoding (hue + position)	Bar color by direction (increase / decrease / unchanged) with a distinct palette; clearer zero line.
WCAG-friendly contrast on dark theme	Central `apply_dashboard_theme()`: tick color `#B0B3B8`, titles `#E8EAED`, subtle grids.
Consistent styling across figures	Shared theme function for line, bar, area, and heatmap; `plotly_dark` template and transparent plot backgrounds.
Document units everywhere	Metric `help=` tooltips; captions explaining YoY on filtered rows; heatmap colorbar title.
Closure / volume for the total series	Filled area under the line via `go.Scatter` with `fill=tozeroy` (avoids `px.line` lacking `fill` in some Plotly versions).
Reduce line-chart noise	Line-only trace (no per-year markers) for the annual total series.
Streamlit chrome vs. data colors	`.streamlit/config.toml`: dark base theme and primary accent aligned with chart cyan (`#38BDF8`) to reduce clash with KPI delta green.
Deprecation fix	`st.plotly_chart(..., width="stretch")` instead of `use_container_width`.

Table 1: Mapping from analysis checklist to implemented revisions.

6 What is better after principles integration and revision?

The revisions were not only cosmetic: they address the gaps identified in Section 3 and make the dashboard **easier to read**, **less ambiguous**, and **more accessible** on a dark theme. Table 1 lists *what* changed; here we state *why* the post-revision interface is stronger.

6.1 User-visible improvements (benefits)

- **Faster reading of the main trend (Gestalt closure):** The filled area under the annual total reinforces "volume over time" at a glance; removing dense per-year point markers reduces clutter so the long-run pattern stands out.
- **Clearer year-over-year story (Bertin redundant encoding):** Positive and negative changes are distinguishable both by bar direction *and* by color, so users do not rely on a single channel; the zero baseline is visually sharper.
- **Less confusion between UI chrome and data (Gestalt similarity):** Aligning Streamlit's primary accent with the chart palette (`config.toml`) reduces the chance that control colors are mistaken for "bad" or "negative" data, compared with a stronger mismatch between red UI accents and green KPI deltas.
- **Explicit meaning of numbers (annotation):** Chart subtitles, axis titles with units (*count per year*), KPI tooltips, and the YoY caption explain what `total` means and why the first filtered year has no YoY bar—reducing misinterpretation.
- **More comfortable reading on dark backgrounds (contrast):** Systematic tick and title colors in `apply_dashboard_theme()` improve legibility of axes and legends relative to an ad hoc default styling.

- **Coherent visual language across tabs:** One theme function and shared palette logic (continents, heatmap colorbar) make the dashboard feel like one instrument rather than unrelated charts.

6.2 Mapping principles to outcomes

- **Gestalt:** Revisions strengthened **similarity** (UI vs. data color harmony), **closure** (area fill for totals), and **proximity of meaning** (subtitles and help near the metrics they explain). Layout grouping was already sound and was preserved.
- **Bertin: Position** for time and magnitude was kept; **redundant encoding** was added for YoY sign; **color value** and tick contrast were standardized; unnecessary **mark** noise on the line chart was removed. Definitions and units are now tied explicitly to encodings (labels and tooltips).

7 Conclusion

We built an interactive Japan inbound immigration dashboard in Python (Streamlit + Plotly), analyzed it with Gestalt and Bertin lenses, and applied a focused set of revisions. After integration of these principles, the interface improves in **interpretive clarity** (what each number means), **encoding redundancy** for signed change (YoY), **visual continuity** for the level series (area fill), **legibility** (contrast and typography), and **consistency** across charts and the Streamlit shell. The screenshots `task1-a.png` and `task1-b.png` document the interface before and after these changes; Section 6 summarizes concrete benefits of the revised version.

AI tooling disclosure

Cursor (https://cursor.com) was used as the **AI coding agent** integrated into the editor for this assignment: it assisted with generating, editing, and debugging Python/Streamlit/Plotly code for the dashboard and project files. The LaTeX source for this report was also produced with AI assistance. The author reviewed all outputs, ran the application locally, and is responsible for the final text, figures, and interpretation.

Data and code

Data file: `japan_immigration_statistics_inbound.csv`.
Main script: `dashboard.py`.
Analysis notes: `task1-b-gestalt-bertin-analysis.txt`.